FASE 2 · RELATÓRIO TÉCNICO

Relatório Técnico – Padrões de Projeto

Onde e por que cada padrão GoF foi utilizado no código do PizzaGest

Disciplina: IES300 – Engenharia de Software III · Prof.ª Elisabete Ferreira dos Santos Gomes Silva
Projeto: PizzaGest – Sistema de Gestão de Pedidos para Delivery Local · **Fase 2:** Implementação e Padrões de Projeto
Repositório: github.com/B3rnardi/restaurante-projeto-eng-III (commit `3357f90`, arquivo `pizzagest.py`)
Integrantes: Luiz Miguel Teles, Cauã Vieira Alves, Leonardo Soares Alcebíades, Fernando de Bernardi Camargo

1. Contexto

O PizzaGest é o protótipo funcional do Sistema de Gestão de Pedidos para Delivery Local, concebido na Fase 1. O código (`pizzagest.py`) segue a arquitetura em três camadas definida naquela fase: a **camada de dados** guarda cardápio, usuários e pedidos; a **camada de negócio** concentra as entidades (`Produto`, `Pedido`, `ItemPedido`) e o `ServicoPizzariaController`; e a **camada de apresentação** é um protótipo de console que executa os casos de uso de login, registro de pedido e emissão de relatório. Foram aplicados três padrões GoF, cada um em um ponto em que o sistema tinha um problema concreto de projeto.

Padrão	Categoria	Onde (pizzagest.py)	Por que, em uma frase
Singleton	Criacional	`BaseDadosPizzariaSingleton`, linhas 8–45; usado nas linhas 140, 167 e 173	Uma única base de dados e um único contador de pedidos para todo o sistema.
Factory	Criacional	`ProdutoFactory.create()`, linhas 75–90; usado na linha 182	Criar `Pizza` ou `Bebida` a partir do tipo informado, sem o cliente conhecer as subclasses.
Strategy	Comportamental	`CalculoPrecoStrategy` e implementações, linhas 96–115; contexto `Pedido`, linhas 145–158	Trocar a regra de preço da pizza meio a meio sem alterar a classe `Pedido`.

2. Singleton – base de dados da pizzaria

Onde. Na camada de dados, na classe `BaseDadosPizzariaSingleton`. O método `__new__` (linhas 15–19) cria o objeto apenas na primeira chamada e devolve sempre a mesma instância nas seguintes; `get_instance()` (linhas 30–32) é o ponto de acesso. A instância é usada pelo `Pedido` para gerar seu número (linha 140) e gravar-se ao ser fechado (linha 167), e pelo `ServicoPizzariaController` para login, cadastro e relatório (linha 173).

Por quê. A base de dados e o gerador de número de pedido são recursos que, por regra de negócio, precisam ser únicos. Com várias instâncias, o relatório do administrador não encontraria os pedidos gravados por outro objeto e dois pedidos poderiam receber o mesmo número. O Singleton garante essa unicidade sem obrigar o programa a passar a base manualmente para cada classe e deixa um ponto único para, nas próximas fases, trocar o armazenamento em memória por um banco de dados real. Para não prejudicar os testes, a classe oferece `limpar_dados()`, chamado antes de cada caso de teste.

3. Factory – criação de produtos do cardápio

Onde. Na camada de negócio, na classe `ProdutoFactory` (linhas 75–90). O método estático `create()` recebe o tipo (“PIZZA” ou “BEBIDA”), normaliza o texto e devolve a subclasse correspondente de `Produto`, levantando `ValueError` para tipos vazios ou desconhecidos. O cliente é o método `gerenciar_cadastro_produto()` do controlador (linha 182). Trata-se de uma *Simple Factory*, a forma simplificada do Factory Method do GoF.

Por quê. O tipo do produto chega como texto, informado no cadastro de produtos. Sem a fábrica, o controlador precisaria conhecer todas as subclasses e repetir o mesmo `if/elif` em todo lugar que criasse produtos. Com ela, a decisão de qual classe instanciar fica em um só ponto, o controlador trabalha apenas com a abstração `Produto` e uma nova categoria (ex.: `Sobremesa`) é incluída criando a subclasse e uma linha na fábrica, sem alterar pedidos, relatórios ou controlador.

4. Strategy – preço da pizza meio a meio

Onde. Na camada de negócio. A interface `CalculoPrecoStrategy` (linhas 96–103) declara `calcular_preco_pizza()`; `PrecoMaiorValorStrategy` (linhas 106–109) cobra o sabor mais caro (regra padrão) e `PrecoMediaStrategy` (linhas 112–115) cobra a média dos dois sabores (regra promocional). O contexto é o `Pedido`, que recebe a estratégia no construtor (linha 145, com a regra padrão quando nenhuma é informada), permite trocá-la com `alterar_estrategia()` (linhas 147–148) e a utiliza em `adicionar_pizza_meio_a_meio()` (linha 155).

Por quê. A forma de cobrar a pizza meio a meio é uma regra comercial que muda com promoções. Colocar essas regras como `if/elif` dentro do `Pedido` faria a classe crescer e ser modificada a cada nova promoção. Com o Strategy, cada regra é uma classe pequena e testável, o `Pedido` não sabe qual regra está aplicando e novas regras entram como novas classes, sem alterar o código existente (princípio Aberto/Fechado). O teste CT03 comprova a troca: a mesma pizza Mussarela (R$ 40,00) + Camarão (R$ 70,00) custa R$ 70,00 com a regra padrão e R$ 55,00 com a promocional.

5. Verificação e próximos passos

Os cinco testes automatizados de `test_pizzagest.py` passam (execução com `python -m unittest -v test_pizzagest`), incluindo o CT04, que integra Factory, Strategy, Pedido, Singleton e relatório em um único fluxo. Para a Fase 3, ficam registrados dois pontos de evolução: proteger o Singleton com `threading.Lock` caso o sistema passe a atender requisições simultâneas como API, e substituir o `if/elif` da fábrica por um registro de tipos se o número de categorias de produto aumentar.